M-Kristo App

Daily Devotions Platform — m-kristo.com

System Design Document

Document code	BGT-MKR-SDD-001
Version	1.0
Date	12 August 2026
Status	Draft for Review
Classification	Confidential

Prepared by Philip Steven Chediel

BlueGrid Technologies | www.bluegrid.co.tz | +255 620 636 893

Document Control

Field	Detail
Project	M-Kristo App (m-kristo.com)
Document	System Design Document (SDD)
Document code	BGT-MKR-SDD-001
Version	1.0
Date	12 August 2026
Author	Philip Steven Chediel
Organisation	BlueGrid Technologies
Contact	www.bluegrid.co.tz | +255 620 636 893
Related	BGT-MKR-SRS-001 (System Requirements Specification)

Revision History

Version	Date	Author	Description
1.0	12 August 2026	Philip Steven Chediel	Initial system design baseline.

Table of Contents

1. Introduction

1.1 Purpose

This System Design Document (SDD) describes the software architecture and detailed design of the **M-Kristo App**. It translates the requirements in the SRS (BGT-MKR-SRS-001) into a technical blueprint for implementation.

1.2 Scope

The design covers the React Native (Expo) mobile client, the Django backend, the data model, API contracts, security, notifications, subscription gating, sharing and deployment.

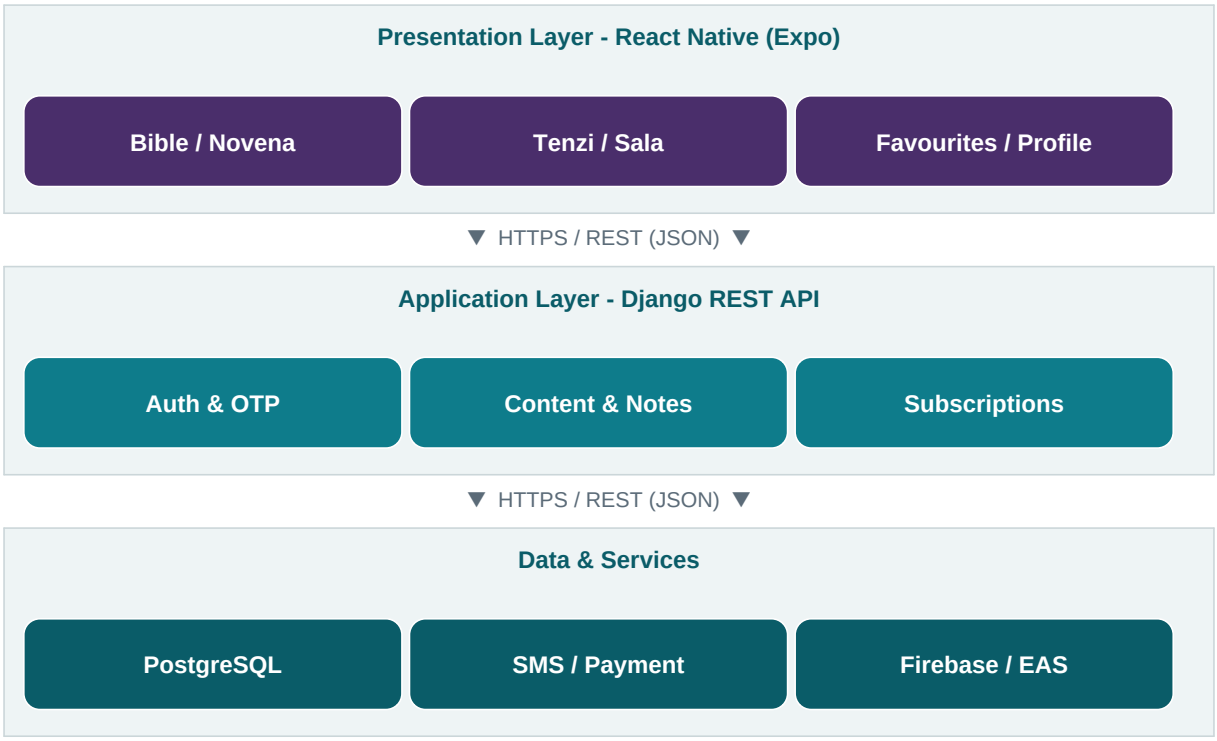
1.3 Design Goals

- Modern, responsive, bilingual (Swahili/English) mobile experience.
- Clear separation between presentation, application and data layers.
- Secure authentication with phone (OTP) verification and social login.
- Reliable offline reading and fast content delivery.
- Clean subscription gating for premium content.
- Maintainable, testable, and CI/CD-friendly codebase.

2. System Architecture

2.1 Architectural Overview

M-Kristo follows a classic three-tier client-server architecture. The mobile client renders the UI and caches content for offline use; the Django backend exposes REST APIs and enforces business rules; a data and services tier holds persistent data and integrates external providers (SMS, payments, push).



2.2 Architectural Style

- Client: component-based React Native with feature modules and a shared design system.
- Backend: Django with a REST API layer, service classes and an authentication/subscription middleware.
- Stateless API using token-based authentication so the backend can scale horizontally.

3. Technology Stack

Layer	Technology	Notes
Mobile client	React Native (Expo)	Single codebase for Android and iOS; EAS build pipeline.
Backend	Django (Python)	REST API, business logic, admin back-office.
Database	PostgreSQL	Relational store for users, content, notes and subscriptions.
Auth	Token / social providers	Email+password, Google, Apple; SMS OTP for phone verification.
Push	Firebase Cloud Messaging or Expo (EAS) Push	Final choice confirmed in design; both supported patterns.
Payments	Third-party payment provider	Monthly and annual subscription billing.
Localization	i18n (Swahili default, English)	Runtime language switch, persisted per user.

4. Module Design

The client is organised into feature modules matching the navigation: Bible, Novena, Tenzi, Sala, Favourites and Profile, plus supporting modules for Auth, Notes, Daily Content and Subscription.

Module	Responsibility	Key requirements
Auth	Signup, OTP verification, login, social login, logout, token handling.	FR-1..FR-6
Bible	Book/chapter/verse navigation, search, offline cache, reading settings.	FR-7..FR-10
Daily Content	Neno la Leo, Tafakari, Somo, Sala and reminders.	FR-11..FR-13
Premium	Shajara, Tenzi, Novena with subscription gating.	FR-14..FR-17
Notes	Create/edit/delete notes; link to scripture.	FR-18..FR-19
Favourites	Save, list and remove favourite items.	FR-20..FR-21
Profile	View/edit info, picture, password, theme, language.	FR-22..FR-26
Sharing	Native Android/iOS share sheet.	FR-28
Notifications	Register device, deliver daily/reminder pushes.	FR-29
Subscription	Plan selection, purchase, status, gating.	SR-1..SR-6

5. Data Design

5.1 Core Entities

Entity	Key fields	Description
User	id, name, email, phone, phone_verified, profile_picture, language, theme, password_hash	Registered application user.
Subscription	id, user_id, plan (monthly/annual), status, start_date, end_date	A user's subscription record.
Content	id, type (bible/neno/tafakari/somo/sala/shajara/tenzi/novena), language, title, body, publish_date, is_premium	All readable content items.
Note	id, user_id, content_ref, text, created_at, updated_at	A user's personal note.
Favourite	id, user_id, content_ref, created_at	A saved item for a user.
Reminder	id, user_id, time, type, enabled	Prayer/daily reminder configuration.
Device	id, user_id, push_token, platform	Registered device for push notifications.

5.2 Relationships

- A User has one active Subscription (0..1) and a history of many.
- A User has many Notes, Favourites, Reminders and Devices.
- A Favourite and a Note reference a Content item.
- Content flagged *is_premium* is served only to active subscribers.

6. API Design

The backend exposes versioned REST endpoints under **/api/v1/**. All responses are JSON and protected endpoints require a bearer token.

Method & path	Purpose	Auth
POST /auth/signup	Create account.	No
POST /auth/verify-otp	Verify phone with OTP code.	No
POST /auth/login	Email/password login.	No
POST /auth/social	Google/Apple login.	No
POST /auth/password-reset	Request/confirm reset.	No
GET /bible	List books/chapters; read verses.	Yes
GET /daily	Neno la Leo, Tafakari, Somo, Sala.	Yes
GET /premium/{type}	Shajara, Tenzi, Novena.	Yes*
GET/POST /notes	List/create notes.	Yes
PUT/DELETE /notes/{id}	Edit/delete a note.	Yes
GET/POST /favourites	List/add favourites.	Yes
DELETE /favourites/{id}	Remove a favourite.	Yes
GET/PUT /profile	View/update profile & settings.	Yes
GET/POST /subscription	View/purchase a plan.	Yes
POST /devices	Register push token.	Yes

* Premium endpoints additionally require an active subscription.

7. Authentication & Security Design

7.1 Signup & Phone Verification Flow

- User submits signup details to **/auth/signup**.
- Backend creates a pending account and sends an OTP via the SMS gateway.
- User submits the code to **/auth/verify-otp**; on success the account is activated and tokens are issued.
- OTP codes are short-lived, single-use and rate-limited.

7.2 Login & Sessions

- Email/password and social (Google, Apple) login issue access and refresh tokens.
- Tokens are stored in secure device storage (Keychain/Keystore).
- Refresh tokens rotate; access tokens are short-lived.

7.3 Security Controls

- All traffic over HTTPS/TLS.
- Passwords hashed with a strong algorithm (Django default).
- Server-side validation of subscription status before serving premium content.
- Input validation and rate limiting on authentication endpoints.

8. Notifications, Subscriptions & Sharing

8.1 Push Notifications

Devices register a push token on login. The backend schedules daily content and reminder notifications and dispatches them through **Firebase Cloud Messaging** or **Expo (EAS) Push**. Reminder times set by the user drive per-user scheduling.

8.2 Subscription Gating

When a user opens premium content the client checks cached subscription status and the backend re-validates on the protected endpoint. Non-subscribers see a locked state and a subscribe prompt offering the monthly or annual plan. Access is revoked automatically when a subscription lapses.

8.3 App Sharing

Sharing uses the native share module so the user can pick any target app (WhatsApp, SMS, email, etc.) through the standard Android and iOS share sheets, sharing an install link to the app.

9. Deployment & Environments

Concern	Approach
Mobile builds	Expo EAS build and submit pipelines for Google Play and the Apple App Store.
Backend hosting	Django served behind HTTPS on a Linux environment with a managed PostgreSQL database.
Environments	Development, staging and production with separate configuration and secrets.
CI/CD	Automated builds and tests; over-the-air updates for the client where applicable.
Monitoring	Application logging, crash reporting and uptime monitoring.

10. Non-Functional Design Considerations

- **Performance:** content caching and pagination keep screens fast on low-bandwidth networks.
- **Offline:** Bible and saved content sync locally for offline reading.
- **Scalability:** stateless APIs and connection pooling support horizontal scaling.
- **Localization:** centralised i18n resources with Swahili default and English support.
- **Accessibility:** scalable text, sufficient contrast and clear navigation.
- **Maintainability:** modular features, documented APIs and automated pipelines.

Prepared by Philip Steven Chediel, BlueGrid Technologies — www.bluegrid.co.tz — +255 620 636 893.